

Week 7 Tutorial

Welcome to the Week 7 tutorial. Today is the first day we will use the popular creative coding library called p5.js. With guidance from your tutor, you will complete the following exercises:

- Set up p5.js in Visual Studio Code
- Display an image using p5.js, make it interactive and animate it
- Create a 'random walker' generative artwork

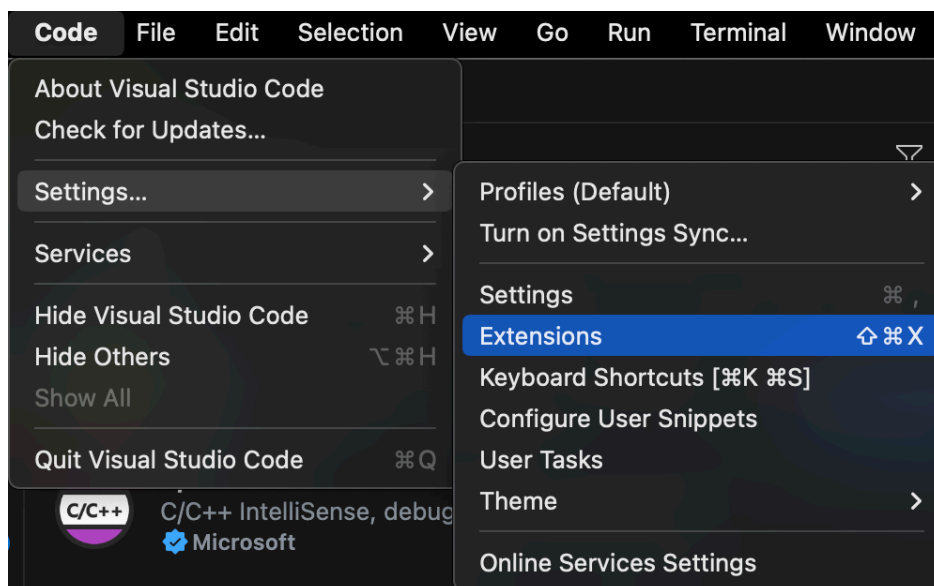
1. Set up p5.js in Visual Studio Code

Using the p5.vscode plugin

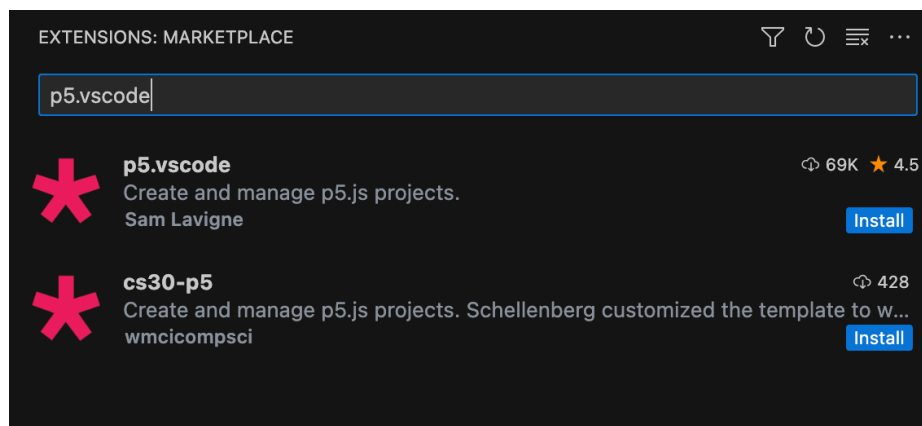
We will be using Visual Studio Code to work with p5.js. To make things easier, we will use the p5.vscode plugin which creates a template to start our projects with.

WARNING: There are other ways of working with p5.js such as using the [p5.js online editor](https://editor.p5js.org/)  (<https://editor.p5js.org/>), however, the quiz and major project submissions will only be accepted as a ZIP file containing files and folders that can be opened and run using VS Code. The ZIP file must contain all the required files to run your program.

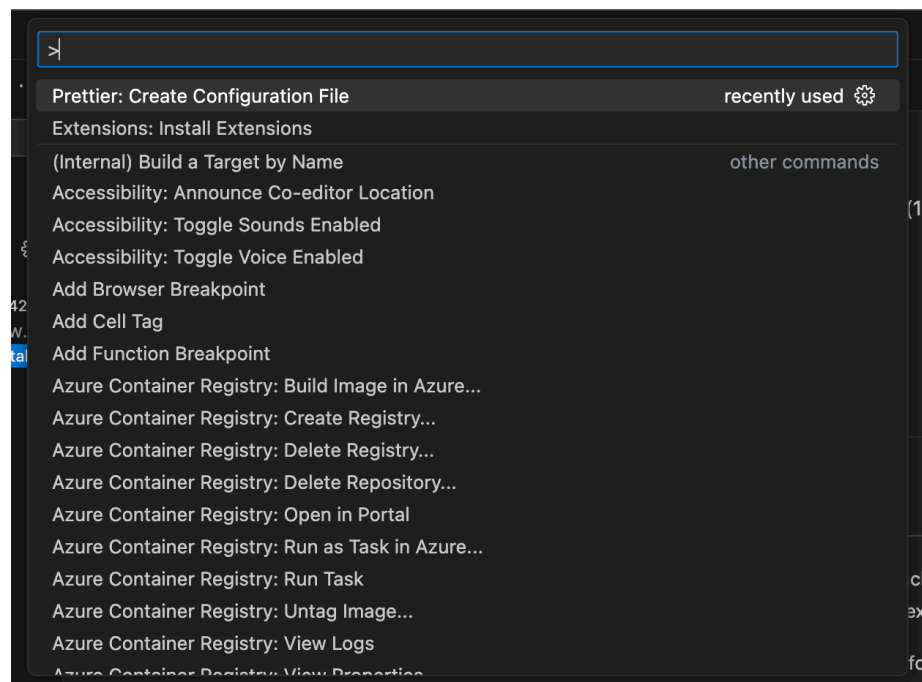
1. Open the Extensions manager in p5.js



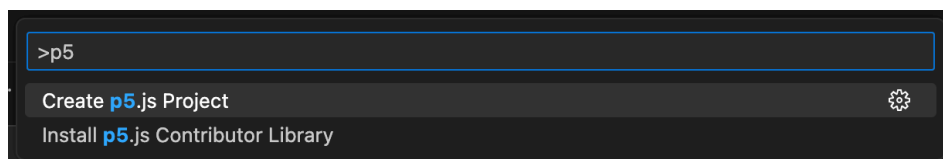
2. Search for the p5.vscode extension and install it



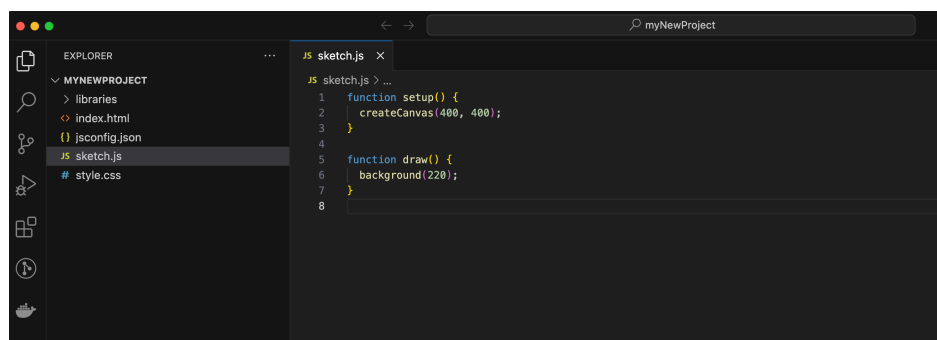
3. Open the Command Palette (with **COMMAND-SHIFT-P** on Mac or **CONTROL-SHIFT-P** on Windows)



4. Choose Create p5.js Project, and select a folder for your project



5. Start coding with your new blank template



2. Display an image using p5.js, make it interactive and animate it

We are going to use the p5.js library to load and display an image. We will add interactivity to allow the user to control where the image is displayed, and animate it so that it moves around the canvas.

2.1. Load and display an image

Objective:

Create an empty p5.js project, which includes an empty `sketch.js` file. Update the sketch to load and display the example image.

Challenge:

Loading an image can take time, particularly for larger images, so the p5.js library implements the `loadImage()` function as an asynchronous function. An asynchronous function is one where the lines of code that follow a call to it are allowed to proceed even if the function has not finished running yet. In the case of `loadImage()`, this means the image might still be loading when subsequent lines of code are run. This is desirable in some situations but we don't want that behaviour here as we want to display the image after it has been loaded.

Desired Result:

We want to first load an image and then have it display in the centre of the screen every frame.

Solution:

The `setup()` function is called once by p5.js at the beginning when running a sketch. It is normally used to initialise properties for our sketch and to create the canvas using `createCanvas()`.

To ensure that the image has completed loading before we display it, we call `loadImage()` inside a special function called `preload()`. The p5.js library calls `preload()` directly before calling `setup()`. Its purpose is to ensure any external files have finished loading. It does this by handling the asynchronous loading of external files in a blocking way. This means no subsequent lines of code can run until it has finished loading.

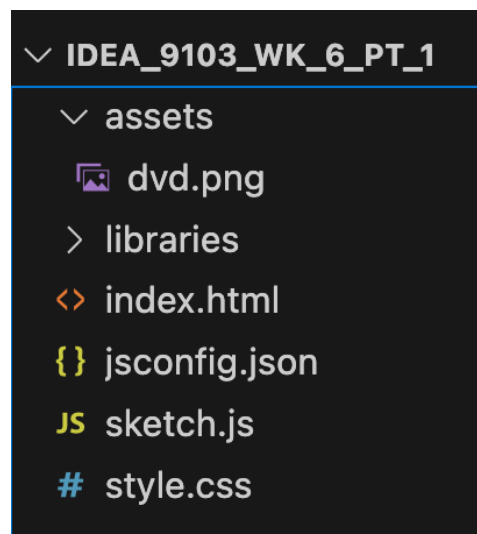
To display the image every frame, the `image()` function is called inside the `draw()` function. The `image()` function calculates the x-y position to centre the image in the middle of the canvas. Half the canvas width and half the logo width are used to centre the image in the x direction. Half the canvas height and half the logo height are used to centre it in the y direction. A set width and height of 100 and 45 is used, which are both a tenth of the original dimensions, preserving the aspect ratio. Directly before calling `image()`, a call to `background()` is made to clear the canvas.

p5.js calls the `preload()` and `setup()` functions once at the start and then calls the `draw()` function repeatedly, once per frame. So, if the current frame rate is 60 frames per second, `draw()` will be called 60 times each second.

Implementation:

Using the p5.vscode extension (see the steps above), make a new p5.js project (i.e. selecting "Create p5.js Project" from the Command Palette). Using the VS Code explorer, create a new subfolder in the project called assets. Click on the following link to download the sample image: [dvd.png \(https://canvas.sydney.edu.au/courses/72128/files/47944030\)](https://canvas.sydney.edu.au/courses/72128/files/47944030). Move the file into the newly created assets subfolder.

Your project should now look like this:



Now let's load the image and display it on the screen with the following code in sketch.js:

```
// This is global variable to hold our image
let logoImage;
// These variables hold the width and height of the image
let logoWidth, logoHeight;

// We will use the preload function, this runs before anything else and it blocks activity until all assets are done loading
function preload() {
  // We need to load the image from disk
  logoImage = loadImage("assets/dvd.png");
}

function setup() {
  createCanvas(800, 600);
  // Set the image dimensions
  logoWidth = 100;
  logoHeight = 45;
}

function draw() {
  // Set the background to blue
  background(0, 0, 255);
  // Draw the image in the centre of the canvas, offsetting the image by half its width and height
  image(logoImage, (width / 2) - (logoWidth / 2), (height / 2) - (logoHeight / 2), logoWidth, logoHeight);
}
```

2.2. Basic interaction and modal operation

Objective:

Add interactivity by making the image follow the mouse cursor. The interactivity will be toggleable using the keyboard.

Desired Result:

When the '2' key is pressed, the logo will follow the mouse cursor around the screen up to the edges of the canvas. When the '1' key is pressed, the logo will return to the centre of the canvas and remain stationary until '2' is pressed again.

Solution:

A switch statement will be used inside the `draw()` function to make the program behave differently depending which operation mode is active.

If the operation mode is “interactive”, the `mouseX` and `mouseY` variables are used in the place of the x and y calculations that centre the image in “still” mode.

Note that if you temporarily comment out the call to `background()` inside `draw()`, you can “paint” the DVD logo across the canvas when you move the mouse cursor around. This shows the role of the `background()` function.

To set the operation mode, the built-in function `keyPressed()` is used. If a '1' is pressed, the operation mode is set to “still” and if a '2' is pressed, the operation mode is set to “interactive”.

Implementation:

Let's add the the interactive mode now using a switch case. Using a switch statement makes it easier to add other operation modes later.

The p5.js parameters `mouseX` and `mouseY` are used to set the logo's position in "interactive" mode.

```
// This is global variable to hold our image
let logoImage;
// These variables hold the width and height of the image
let logoWidth, logoHeight;
// This variable will hold the operation mode of the program
let operationMode = "still";

function preload(){
  // We need to load the image from disk
  logoImage = loadImage("assets/dvd.png");
}

function setup() {
  createCanvas(800, 600);

  // Set the image dimensions
  logoWidth = 100;
  logoHeight = 45;
}

function draw() {
```

```
// Set the background to blue
background(0, 0, 255);

// We will use a switch statement to check the operation mode, and draw the image accordingly
switch (operationMode) {
  case "still":
    // Draw the image in the centre of the canvas, offsetting the image by half of its width
    // and height
    image(logoImage, (width / 2) - (logoWidth / 2), (height / 2) - (logoHeight / 2), logoWidth, logoHeight);
    break;
  case "interactive":
    // Display the DVD logo and follow the mouse position
    image(logoImage, mouseX, mouseY, logoWidth, logoHeight);
    break;
}

// We will use a key press to change the operation mode
function keyPressed() {
  if (key == "1") {
    operationMode = "still";
  }
  if (key == "2") {
    operationMode = "interactive";
  }
}
```

2.3. Animation

Objective:

Make the image move automatically, so that when it reaches the edge of the screen it bounces away in the opposite direction.

Challenge:

We need to make the image move automatically across the screen.

We also need to reverse the direction of the DVD logo when it reaches the edge of the screen. This problem is easier to solve if it is broken into two steps:

1. Detect when the image has reached the edge of the screen.
2. Reverse the direction of the image accordingly.

Desired Result:

When the '3' key is pressed, the logo will begin to animate, moving towards the edges of the canvas. When the logo reaches the edge of the canvas, it should bounce away and move in the opposite direction.

Solution:

Firstly, animate the DVD logo by changing the position it is drawn at every frame. Because the `draw()` function is called every frame (60 times per second), if we increment the draw position by a small amount each frame, the image will move. If the first position of the image on the x axis is

100, and we increment it by 5 each frame, each frame it will move to the right by 5 pixels. If instead we subtract 5 from the position each frame it will move to the left. The more we increment the draw positions each frame, the faster the image will move, therefore we will use speedX and SpeedY variables to increment the position

Next, check the x-position and y-position independently. If the x-position or y-position extends beyond the bounds of the canvas (accounting for the width and height of the logo) then reverse the speed of the logo (this will make it move in the opposite direction). We will do this reversing independently for each axis. If the x-position extends too far to the left or right, reverse the x-speed (multiply it by -1). If the y-position extends too far to the top or bottom of the canvas, reverse the y-speed (multiply it by -1). The bounds of the canvas can be determined by using 0 for the left and top edges, and width and height for the right and bottom edges, respectively.

Implementation:

Now we are going to add the animation mode to the sketch. The logo will move in a linear direction and will bounce back off the edges if it hits any of the four sides of the canvas:

```
// This is global variable to hold our image
let logoImage;
// Two variables to hold the width and height of the image
let logoWidth, logoHeight;
// This variable will hold the operation mode of the program
let operationMode = "still";

// Variables to hold the position and speed of the logo when we animate it
let xPos, yPos;
let xSpeed, ySpeed;

function preload(){
  // We need to load the image from disk
  logoImage = loadImage("assets/dvd.png");
}

function setup() {
  createCanvas(800, 600);

  // Set the image dimensions
  logoWidth = 100;
  logoHeight = 45;

  // Set initial position randomly
  xPos = random(width);
  yPos = random(height);

  // Set speed values in pixels per frame
  xSpeed = 4;
  ySpeed = 4;
}

function draw() {
  // Set the background to blue
  background(0, 0, 255);

  // We will use a switch statement to check the operation mode, and draw the image accordingly
  switch (operationMode) {
    case "still":
      // Draw the image in the centre of the canvas, offsetting the image by half of its width
      // and height
      image(logoImage, (width / 2) - (logoWidth / 2), (height / 2) - (logoHeight / 2), logoWidth, logoHeight);
      break;
  }
}
```

```

    case "interactive":
        // Display the DVD logo and follow the mouse position
        image(logoImage, mouseX, mouseY, logoWidth, logoHeight);
        break;
    case "moving":
        // Move the DVD logo according to its speed (if the speed is negative, the logo will move
        // in the opposite direction)
        xPos += xSpeed;
        yPos += ySpeed;

        // Check for collision with edges and reverse speed if collision detected
        // We need to check the right edge with the width of the canvas plus the width of the logo,
        // and the left edge with 0
        if (xPos + logoWidth >= width || xPos <= 0) {
            // If there is a collision in this direction, invert the speed so that the logo moves
            // in the opposite direction
            xSpeed = -xSpeed;
        }
        // Use the same logic to check the top and bottom edges for a collision
        if (yPos + logoHeight >= height || yPos <= 0) {
            // If there is a collision in this direction, invert the speed so that the logo moves
            // in the opposite direction
            ySpeed = -ySpeed;
        }
        // Display the DVD logo using the updated positions
        image(logoImage, xPos, yPos, logoWidth, logoHeight);
        break;
    }
}

// We will use a key press to change the operation mode
function keyPressed() {
    if (key == "1") {
        operationMode = "still";
    }
    if (key == "2") {
        operationMode = "interactive";
    }
    if (key == "3") {
        operationMode = "moving";
    }
}
}

```

3. Create a 'random walker' generative artwork

You have all made some great efforts at producing aesthetically pleasing images and designs with code. Making more interesting and surprising images and designs leveraging p5.js will be the focus of the next part of this course. We are going to start with a classic generative effect that uses the 'random walker' algorithm. While simple, this can be the starting point for making a number of interesting generative artworks.

3.1. Basic random walker

Objective:

We will make a random walker – it is a simple system where we draw a line between two random points, then draw a new line from the end point of the previous line to a new random point. This continues, creating a web of connected lines.

Challenge:

For the effect to work, we need to keep the previously drawn lines on the canvas as we draw them. We could do this by storing all the lines' points in an expanding array and displaying all the lines each frame. Alternatively we could simply choose to not clear the canvas each frame so the lines accumulate each frame. We will take the latter approach.

Desired Result:

We want to create a colourful artwork composed of randomly positioned lines that are drawn over time. Starting from the centre of the canvas, a coloured line will be drawn towards a random end point. After each line is drawn, the colour will change at random. However, the amount of variation in colour is constrained so that it appears to be a gradual change.

Solution:

The random walker algorithm draws a line starting at the centre of the canvas and ending at a point that is determined using random numbers. The random numbers are limited to the range defined as (`-maxLineLength` , `maxLineLength`). The thickness of the line is defined by `lineWeight` .

After each line is drawn the colour is varied. The variation is limited to the range defined as (-10, 10). This limited range has the effect of making it appear as if the colour is gradually varying over time. As we do this operation continuously, we need to make sure the values stay in between 0 and 255 - the limits of RGB colour values. To do this we use the `constrain()` function, (`constrain()` is a built-in p5.js), this function will clamp an incoming value inside the given range. In this case if a value of 270 is passed into the constrain function it will output 255, if a value of -10 is passed to the value it will output 0. Due to the random range of -10 to 10 for each colour change, the colour wont get locked at either end of the spectrum.

Note that `background()` is called once in the `setup()` function but, unlike in the previous example, it is not called in the `draw()` function. This is so the lines accumulate on the canvas over time, building up an artwork made from randomly positioned lines.

Implementation:

The code below implements the 'random walker' algorithm outlined above:

```
// Declare global variables for x and y coordinates
let xPos;
let yPos;

// Let's make a variable to control the maximum line length
let maxLineLength = 30;

// Declare global variables for RGB colour values
let r;
let g;
let b;

// The setup function runs once when the program starts
function setup() {
  // Create a canvas of size 500x500 pixels
  createCanvas(500, 500);

  // Initialise x and y to the centre of the canvas
  xPos = width / 2;
  yPos = height / 2;
```

```

// Randomly set the initial RGB values between 0 and 255
r = random(255);
g = random(255);
b = random(255);

// Set the background colour of the canvas to a dark grey (32), this is happening in the set
up function so it only happens once
background(32);
}

// The draw function runs continuously after setup
function draw(){
  // Note, there is no background() function here, so the lines will be drawn on top of each o
  ther.
  // If we want to clear the canvas we need to use the clear() function
  // or set the background to a colour in the draw function
  // Run the drawRandomLine function a single time each frame
  // We can control the speed of the animation by changing the number of iterations
  for(let i = 0; i < 1; i++){
    drawRandomLine();
  }
}

// Custom function to draw lines with random colours and directions
function drawRandomLine() {

  // Calculate the next x and y coordinates by adding a random value between -20 and 20 to the
  current x and y
  let nextX = xPos + random(-maxLineLength, maxLineLength);
  let nextY = yPos + random(-maxLineLength, maxLineLength);

  // Ensure the nextX and nextY values stay within the canvas boundaries
  // We use the constrain function to do this
  // If the value is less than the min value parameter, it will be set to the min input parame
  ter
  // If it is greater than the max value parameter, it will be set to the max value parameter

  nextX = constrain(nextX, 0, width);
  nextY = constrain(nextY, 0, height);

  // We get the last R, G and B values and add a random value between -10 and 10 so they just
  change a little
  r += random(-10, 10);
  g += random(-10, 10);
  b += random(-10, 10);

  // Ensure the RGB values stay within the range of 0 to 255 again using the constrain functio
  n
  r = constrain(r, 0, 255);
  g = constrain(g, 0, 255);
  b = constrain(b, 0, 255);

  // Set the stroke colour using the RGB values
  stroke(r, g, b);

  // Draw a line from the current x,y to the next x,y
  line(xPos, yPos, nextX, nextY);

  // Update the current x and y coordinates to the next x and y coordinates
  // This is so the next line will start from the end of the previous line
  xPos = nextX;
  yPos = nextY;
}

```

3.2. Controlling the style in real time

Objective:

In this second part, we will add interactivity to the previous sketch. The interactivity lets us control the generative artwork by changing the maximum line lengths, changing the line width, and clearing the canvas.

Challenge:

To refine the look of our artworks, it's important for us to be able to dynamically adjust the parameters we use for rendering to the screen. This will allow us to create intricate and evolving designs that respond to various stimuli. Your assignment will focus on this responsive aspect, and your designs will respond to different inputs like timers, user interactions, music, or random numbers.

When creating responsive or interactive designs, it's important to be able to test out the connection between inputs (like user interaction) and parameters that influence our output (like colours, sizes of drawings or speed of movements). This lets us observe the effects of variable adjustments in real-time. An easy way to do this is to control values with keyboard inputs, allowing us to tweak our sketch on-the-fly and instantly see the results.

Desired Result:

In this version, keyboard interaction will be used to modify the line length and line weight as the artwork is drawn. The line length can be increased or decreased using the up and down arrows. The line weight can be increased or decreased using the right and left arrows, respectively. The current values of `lineWeight` and `maxLineLength` will be displayed on screen.

Solution:

The `keyPressed()` function is used to determine which keys were pressed. Pressing the 'c' key will clear the artwork.

Adjustable line length and width is achieved by adding or subtracting a fixed amount to either `lineWeight` or `maxLineLength` each time the relevant arrow key is pressed.

We will use the `constrain()` function again to ensure the `lineWeight` and `maxLineLength` variables don't extend past a reasonable limit.

The current values for `lineWeight` or `maxLineLength` are displayed to the screen using the `text()` function. We need to draw a rectangle behind this text because we are not clearing the screen each frame. If we didn't do this the text would accumulate on top of itself each frame. When we draw a rectangle behind it, the rectangle is drawn on top of the old text, and then the new text is drawn on top of that.

Implementation:

In the next code block, we'll enhance our previous sketch by introducing controls for line width and maximum line length. This will let us see how modifications in these parameters influence the visual output in real-time:

```
// Declare global variables for x and y coordinates
let xPos;
```

```

let yPos;

// Let's make a variable to control the maximum line length
let maxLineLength = 30;

// Declare global variables for RGB colour values
let r;
let g;
let b;

// Declare global variables for the width and height of the drawing area (we will not use the
whole canvas)
let drawWidth = 500;
let drawHeight = 500;

// Let's make a variable to control the line weight
let lineWeight = 1;

// The setup function runs once when the program starts
function setup() {
  // Create a canvas of size 500x600 pixels
  createCanvas(500, 600);

  // Initialise x and y to the centre of the canvas
  xPos = drawWidth / 2;
  yPos = drawHeight / 2;

  // Randomly set the initial RGB values between 0 and 255
  r = random(255);
  g = random(255);
  b = random(255);

  // Set the background colour of the canvas to a dark grey (32), this is happening in the set
up function so it only happens once
  background(32);
}

// The draw function runs continuously after setup
function draw(){
  // Note, there is no background() function here, so the lines will be drawn on top of each o
ther.
  // If we want to clear the canvas we need to use the clear() function
  // or set the background to a colour in the draw function
  // Run the drawRandomLine function a single time each frame
  // We can control the speed of the animation by changing the number of iterations
  for(let i = 0; i < 1; i++){
    drawRandomLine();
  }
}

// Custom function to draw lines with random colours and directions
function drawRandomLine() {

  // Calculate the next x and y coordinates by adding a random value between -20 and 20 to the
current x and y
  let nextX = xPos + random(-maxLineLength, maxLineLength);
  let nextY = yPos + random(-maxLineLength, maxLineLength);

  // Ensure the nextX and nextY values stay within the canvas boundaries
  // We use the constrain function to do this
  // If the value is less than the min value parameter, it will be set to the min input parame
ter
  // If it is greater than the max value parameter, it will be set to the max value parameter
  nextX = constrain(nextX, 0, drawWidth);
  nextY = constrain(nextY, 0, drawHeight);

  // We get the last R, G and B values and add a random value between -10 and 10 so they just
change a little
  r += random(-10, 10);
  g += random(-10, 10);
  b += random(-10, 10);

```

```

// Ensure the RGB values stay within the range of 0 to 255 again using the constrain function
r = constrain(r, 0, 255);
g = constrain(g, 0, 255);
b = constrain(b, 0, 255);

// Set the stroke colour using the RGB values
stroke(r, g, b);

// Set the stroke weight to lineWidth
strokeWeight(lineWeight);
// Draw a line from the current x,y to the next x,y
line(xPos, yPos, nextX, nextY);

// Update the current x and y coordinates to the next x and y coordinates
// this is so the next line will start from the end of the previous line
xPos = nextX;
yPos = nextY;

// Draw a rectangle as a background for the text
fill(32);
noStroke();
rect(0, drawHeight, drawWidth, height-drawHeight);

// Draw a string with the key commands
fill(255);
// Draw some text to the screen showing the control keys and the current values
text(` Press the up or down arrows to change the max line length. Current Max line length:
${maxLineLength}`, 10, drawHeight + 20);
text(` Press the left or right arrows to change the line weight. Current line weight: ${line
Weight}`, 10, drawHeight + 40)
text(` Press 'c' to clear the canvas`, 10, drawHeight + 60);
}

// The keyPressed function is called once every time a key is pressed
function keyPressed() {
  if (keyCode === UP_ARROW) {
    maxLineLength += 5;
    maxLineLength = constrain(maxLineLength, 2, drawWidth/2);
  }
  if (keyCode === DOWN_ARROW) {
    maxLineLength -= 5;
    maxLineLength = constrain(maxLineLength, 2, drawHeight/2);
  }
  if (keyCode === RIGHT_ARROW) {
    lineWeight += 1;
    lineWeight = constrain(lineWeight, 1, 10);
  }
  if (keyCode === LEFT_ARROW) {
    lineWeight -= 1;
    lineWeight = constrain(lineWeight, 1, 10);
  }
  // if the key is c we clear the canvas
  if (key === 'c') {
    background(32);
  }
}

```

4. Inspiration for the week

This week's inspiration is the work of generative artist (and co-founder of Etsy) Jared Tarbell.

Each of the following examples was created by a drawing process that uses a simple set of rules that runs many times over to build a complex, cohesive and interesting work.

<http://www.complexification.net/gallery/machines/substrate/> 

(<http://www.complexification.net/gallery/machines/substrate/>)

<http://www.complexification.net/gallery/machines/nodeGarden/> 

(<http://www.complexification.net/gallery/machines/nodeGarden/>)

<http://www.complexification.net/gallery/machines/happyPlace/> 

(<http://www.complexification.net/gallery/machines/happyPlace/>)

If you would like to understand more about Jared Tarbell's approach to making art, you can read an interview with him here:

<https://www.artnome.com/news/2020/8/24/interview-with-generative-artist-jared-tarbell> 

(<https://www.artnome.com/news/2020/8/24/interview-with-generative-artist-jared-tarbell>)

In this excerpt from the interview the artist talks about the algorithm used to create Substrate, the first work linked above:

So let's look at the algorithm *Substrate*. You can understand the semantics of that program very easily. Like, this is the rule, it's what's happening. There's a line that goes straight until it hits another line or the edge of the screen, and then new lines form at right angles to existing lines. That's all you need to know. There are just two rules. And then watching the whole system unfold, I think it gives you an idea of the potential depth. It's just this really simple thing that's operating, but look at the complexity of the results that are being generated.

The best algorithms that I've created are the ones where the semantics are easily understood or described. Then you have a deeper appreciation for the results of the algorithm, the resulting image.

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)